4

Extra laboratory 1

4. Generalised Hough transform

4.1 Main objectives:

- implementation of the generalised Hough transform,
- pattern search using the generalised Hough transform.

4.1.1 R-table

4.2 Implementation of the generalised Hough transform

Exercise 4.1 Pattern searching.

1. From the course page, download the data archive for the exercise and unzip it in your own working directory.
2. Create a new script. Based on the image with the pattern `trybik.jpg`, we will create an array `R-table`. To do this, determine the **contours** and **gradients** in the pattern image.
3. In order to **determine the contour**, one has to first perform image conversion to greyscale and binarisation with an appropriate threshold. Next, use the function `cv2.findContours` to get the contours present in the image.

R Inverse the values of the image – `cv2.bitwise_not` – before sending it to the function, because it contains a black contour on a white background, and the function `findContours` expects the opposite.

i	ϕ	R_{ϕ_i}
1	0	$(r_{11}, \alpha_{11})(r_{12}, \alpha_{12}) \dots (r_{1n}, \alpha_{1n})$
2	$\Delta\phi$	$(r_{21}, \alpha_{22})(r_{22}, \alpha_{22}) \dots (r_{2m}, \alpha_{2m})$
3	$2\Delta\phi$	$(r_{31}, \alpha_{31})(r_{32}, \alpha_{32}) \dots (r_{3k}, \alpha_{3k})$
...	...	...

Table 4.1: R-table content.

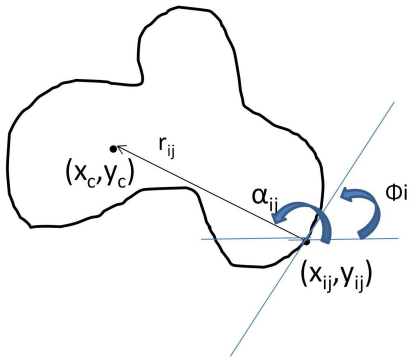

Figure 4.1: R-table calculation. Source: Generalised Hough transform – Wikipedia.

Note to get lists of all contour points – use the parameter `CHAIN_APPROX_NONE`.

R In the case where the contour is split, you can use morphological operations to prevent splits or set the `RETR_TREE` parameter, which arranges the contours from the longest to the shortest and allows you to select the longest contour.

4. Display the contour using a function such as `cv2.drawContours`.
5. Compute the gradients using the **Sobel** filters.
   - Compute the gradient magnitude,
   - Compute the gradient orientation.

   R It is advisable to normalise the gradient magnitude by its maximum value.
6. **Choose a reference point** – let it be the centre of gravity of the pattern determined from a binarised image of the pattern using moments. The central moments of `m00`, `m10`, and `m01` (function `cv2.moments(bin, 1)`) can be used to determine the centre of gravity.
7. Vectors connecting the contour/contours points to the reference point will be needed to fill the `R-table`. Write into `R-table` **the lengths of these vectors** and **the angles** they form with the OX axis.
   The entry point of the `R-table` is determined by the orientation of the gradient at the contour point (determined from Sobel mask filtering) – **convert radians to degrees** – `R-table` will have 360 rows. We use an accuracy of 1 degree.
   Then, for example, `Rtable[30]` will be a list of polar coordinates of the contour points whose orientation calculated from the gradients is about 30°.
8. Using the image `trybiki2.jpg` and the `R-table` from the previous section, fill the two-dimensional Hough space – recalculate the gradient at each point and for points whose normalised gradient value exceeds 0.5, determine the orientation at that point, then increase the value by 1 in the accumulation space at the points stored in `R-table` according to the dependence:

```
x1 = -r*np.cos(fi) + x
y1 = -r*np.sin(fi) + y
```

where r, fi – values from the corresponding row in the `R-table`, x,y – coordinates of the point for which the gradient is greater than 0.5.

The figure of Hough's space is also worth displaying.

9. Search for the maximum in Hough space and mark it on the input image – `trybiki2.jpg`.
10. The result of the operation of the above algorithm is marking the detected maximum on the image and overlaying the template contour around this point. Determine all five maxima in the image.

    R In order to easily find subsequent maxima, it is advisable to first zero out a certain area around an already detected maximum.

11. An example solution is shown in Fig. 4.2, and additionally Fig. 4.3 illustrates the Hough space.

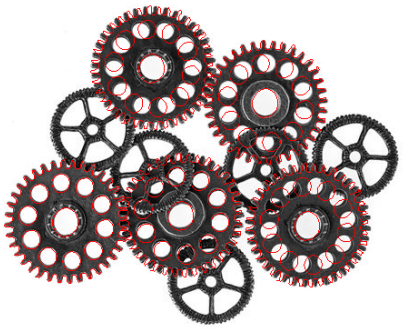

Figure 4.2: Example solution.

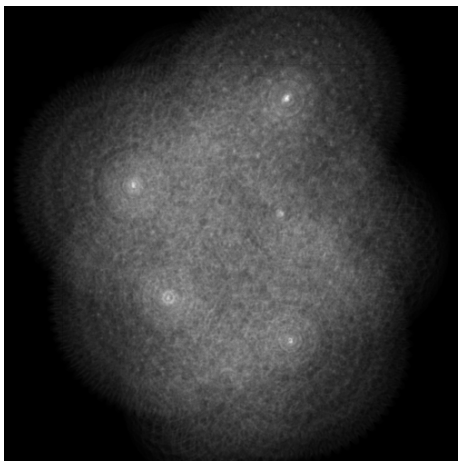

Figure 4.3: Hough space.

4.3 Searching for patterns differing in orientation

Exercise 4.2 The following task presents a method that uses the Hough space to search for patterns that differ in orientation.

1. Based on the algorithm for pattern searching from the basic task, an additional functionality will be introduced that will affect the interpretation of patterns differing in orientation.
2. In the first step, we extend the Hough space to three dimensions by adding rotations. We assume an initial discretisation of the rotation angle of 10 degrees. The following syntax can be used for this purpose:

```
new_hough_shape = hough.shape + (36,)
new_hough = np.zeros(new_hough_shape)
```

   where (36,) is a single-element tuple – it represents the third dimension – an angle step of 10 degrees.
3. Taking rotations into account is done in two steps. In the basic version, we computed the orientation at the point under consideration and, based on that, we referred to the *R-table*. Now, at a given point we will consider 36 orientations – the base one and those shifted by 10 degrees. Thus, the code from point 4.2 should be modified so that the incrementation is carried out for every tenth angle in the range $[0-360)$. In practice, we subtract the angles from the discretised value of the orientation at the point *alpha* and take the modulus, i.e.:

```
alpha_n = |alpha - d_alpha|
```

   where *d_alpha* are the successive values $0, 10, 20, 30, ..., 350$).
4. In the second step, the previously applied change of orientation must be "compensated for". To this end, we read the appropriate value of the angle fi for the pattern from the `R-table` and, when determining the accumulator points, we use the modified formulas from Section 4.2:

```
x1 = -r*np.cos(fi + df) + x
y1 = -r*np.sin(fi + df) + y
```

   where *df* are the successive rotation angles *d_alpha* (but expressed in radians). To detect all patterns in the image, one should in principle detect sufficiently large local maxima. However, for simplicity, we will replace this operation with repeatedly detecting the global maximum and zeroing out its neighbourhood. To find a maximum in the 3D Hough space, one can use the `numpy` array method `argmax()`. However, it returns a single index in an array "flattened" to one dimension. To obtain indices that are useful for us (over 3 dimensions), the result of `argmax()` must be passed to the function `np.unravel_index` (along with the array's `shape`). After finding the maximum, mark it on the input image (only 2 coordinates of the maximum are needed; the angle is not important for now).
5. If the same maximum was found, zero out the neighbourhood of the maximum in the Hough space – for example:

```
hough[m[0]-delta:m[0]+delta, m[1]-delta:m[1]+delta, :] = 0
```

   where m contains the indices of the maximum, and *delta* is the size of the neigh-

bourhood (you can use the value 30). Then repeat the operations from the previous point. Find and mark a total of 5 successive maxima. Check whether the correct objects have been found. In case of problems, consider increasing the angular discretisation resolution, e.g. from 36 to 72.
For better visualisation, the pattern can additionally be drawn at the detected points. To this end, you should plot on the input image the points corresponding to all vectors from the *R-table* array, using formulas very similar to those from Section 4.2, where x, y are the coordinates of the detected maximum, and the angle df follows from the third coordinate of this maximum.

■